Big Data Storage and Retrieval Report

March 18, 2026

1 Abstract

This report presents the work done for the Big Data Storage and Retrieval assignment. The dataset was cleaned, modeled, and loaded into PostgreSQL, MongoDB, and Neo4j. After that, three analysis tasks were implemented. The analysis and benchmark parts were completed for PostgreSQL and MongoDB. Neo4j was modeled, but it was not used for the analysis and benchmark stages because of technical issues during the work. The results show that PostgreSQL gave more stable benchmark numbers, while MongoDB was very strong for the text search task. MongoDB query Q1 was much slower on the test laptop, so its benchmark was run only one time.

2 Introduction

The goal of this assignment was to compare different database models for an e-commerce dataset that contains customer events, campaigns, campaign messages, first purchase dates, and friendship links between users. The work followed three main parts from the assignment:

- data modeling and storage,
- data analysis,
- benchmarking.

The cleaned dataset used in this work contains the following files:

- `campaigns.csv`: 1,907 rows,
- `client_first_purchase_date.csv`: 175,780 rows,
- `events.csv`: 1,277,717 rows,
- `friends.csv`: 1,974,234 rows,
- `messages.csv`: 2,994,126 rows.

The experiments were run on a laptop with Microsoft Windows 11 Pro, an Intel Core i5-9300H CPU at 2.40 GHz, 4 cores and 8 logical processors, and about 8 GB RAM. PostgreSQL and MongoDB were used locally on the machine. Neo4j was also prepared for the modeling task. The exact software version numbers were not saved in the repository during the experiments, so they are not reported here.

3 Data Modeling

3.1 Data Cleaning

The cleaning step was implemented in `scripts/clean_data.py`. The script reads the raw CSV files and writes cleaned files to `data/processed`. The main cleaning actions were:

- trimming spaces,
- replacing common missing tokens such as `na`, `null`, and empty strings with empty values,
- normalizing Boolean values into a consistent form,
- normalizing date and datetime fields,
- normalizing numeric values,
- skipping malformed rows with extra unnamed columns,
- skipping fully empty rows.

This step was important because the same cleaned files were used by all database loaders, which helped make the comparison fair.

3.2 PostgreSQL Data Model

The PostgreSQL model was implemented in `scripts/load_data_psql.sql`. A relational design was chosen because the dataset has clear entities and many relationships that fit well into tables with keys and constraints.

The main tables are:

- `users`,
- `clients`,
- `products`,
- `events`,
- `campaigns`,
- `messages`,
- `friends`.

The loader first creates staging tables, imports the cleaned CSV files, and then fills the final tables. Primary keys, foreign keys, and a unique key on `(campaign_id, campaign_type)` were used to keep the data consistent. This model is normalized and easy to query with joins, which is useful for the analysis tasks.

3.3 MongoDB Data Model

The MongoDB model was implemented in `scripts/load_data_mongodb.js`. A document model was chosen to reduce the need for many joins and to keep related values together inside documents when that made sense.

The main MongoDB collections are:

- `users`,

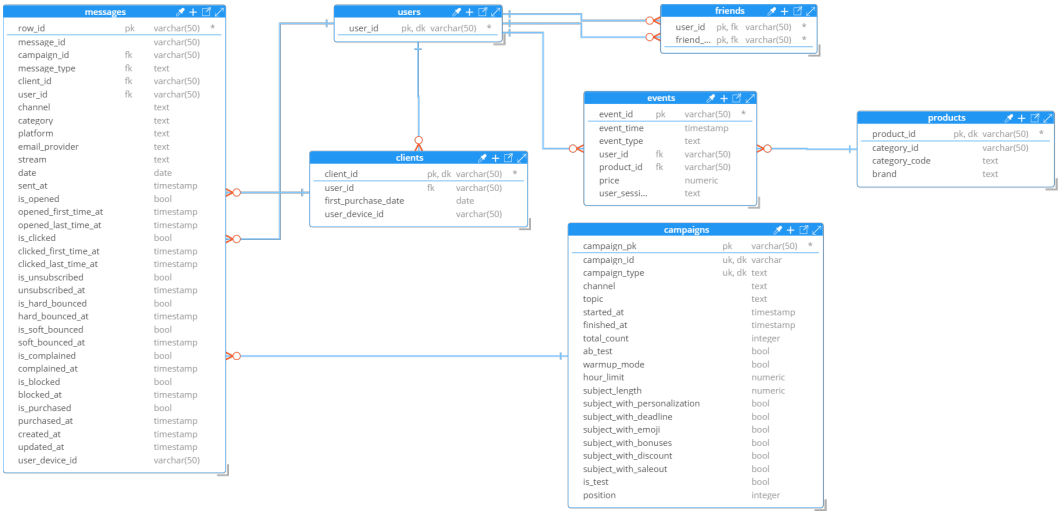


Figure 1: PostgreSQL data model

- `clients`,
- `products`,
- `events`,
- `campaigns`,
- `messages`,
- `friendships`.

Some values were grouped into sub-documents. For example, campaign information inside messages is stored as `campaign_ref`, and message actions are stored in an `engagement` object. Indexes were added for fields such as user IDs, product IDs, campaign keys, and dates. This design helped MongoDB perform well on some tasks, especially the text search task.

3.4 Neo4j Data Model

The graph model was implemented in `scripts/load_data_graph.cypherl`. The design uses graph nodes such as `User`, `Client`, `Campaign`, `Message`, `Product`, and `Category`, with relationships between them.

This model is a good fit for friendship links and recommendation tasks because social connections are first-class relations in a graph database. However, during the work I faced technical issues with Neo4j. Because of that, Neo4j was modeled and prepared, but I did not complete the data analysis and benchmark tasks for it. For this reason, the experimental results in this report focus on PostgreSQL and MongoDB.

4 Experimental Results

4.1 Analysis Tasks

The analysis was implemented with Python clients in:

- `scripts/analyze_psql.py`,

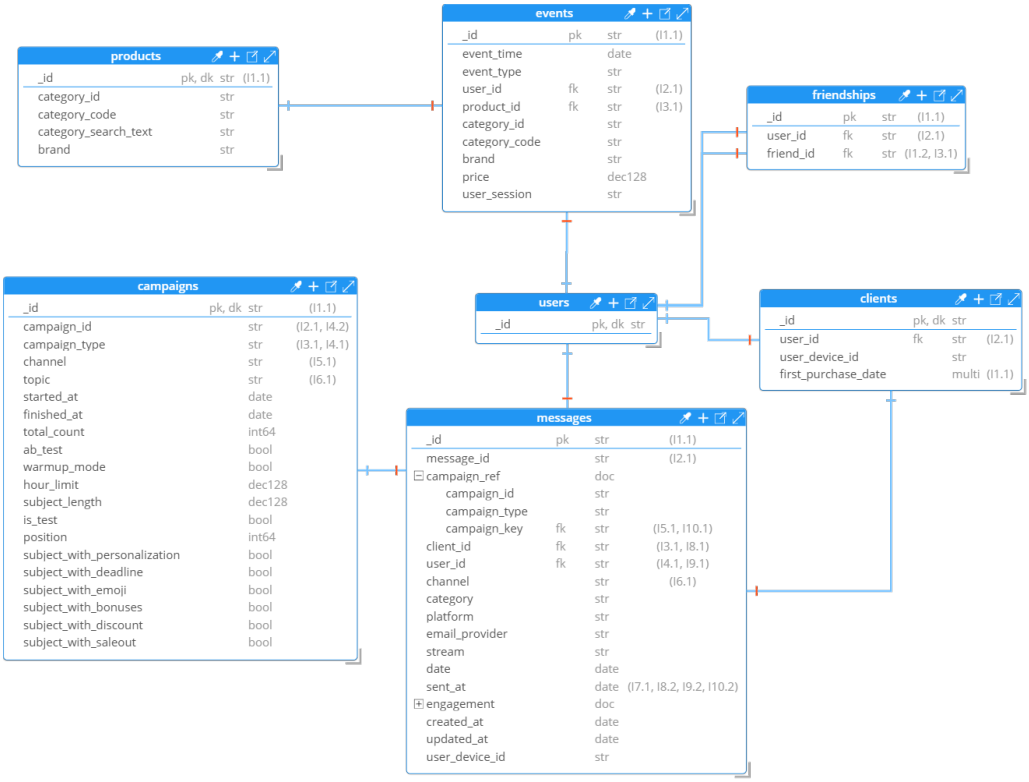


Figure 2: MongoDB data model

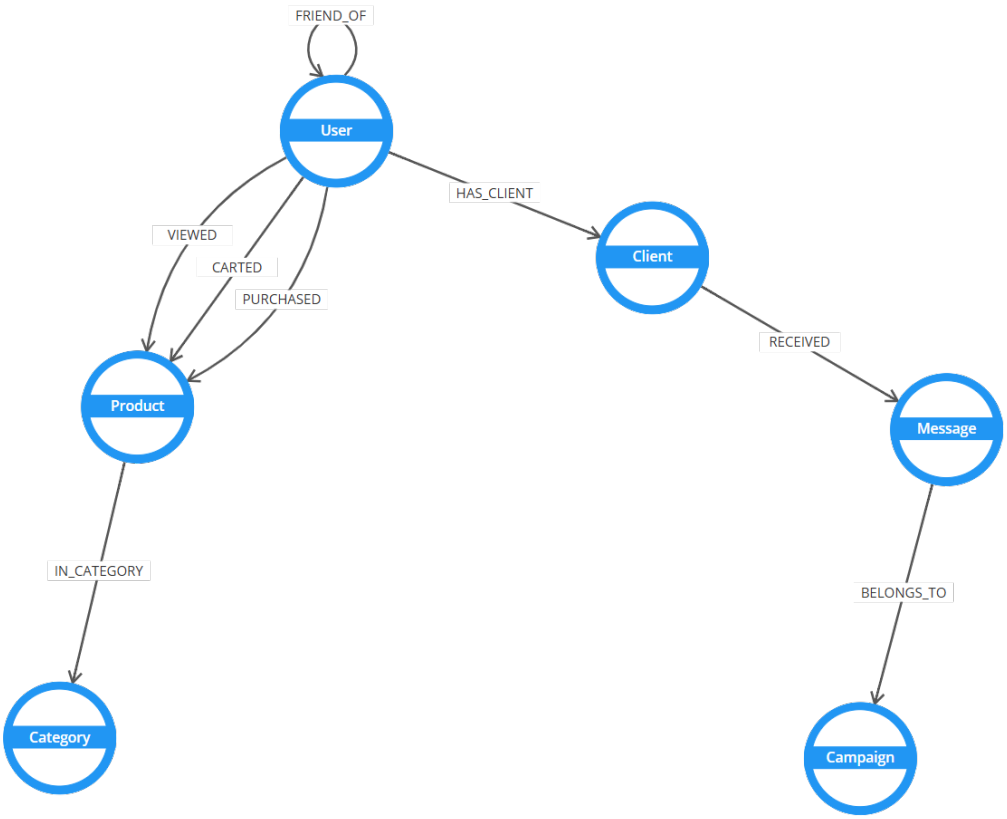


Figure 3: Neo4j data model

- `scripts/analyze_mongodb.py`.

The full query files are:

- `scripts/q1.sql` and `scripts/q1.js`,
- `scripts/q2.sql` and `scripts/q2.js`,
- `scripts/q3.sql` and `scripts/q3.js`.

4.1.1 Q1: Did campaigns attract purchases?

Q1 studies whether campaigns led users to purchases. The query first groups messages by campaign and user, then checks which users purchased. After that, it uses the friendship data to find how many purchasers also had at least one friend who purchased in the same campaign. This gives a social support signal.

Both PostgreSQL and MongoDB returned the same top 20 campaign results. The strongest conversion rates were mostly from transactional email campaigns. The best result was campaign `33` with topic `order pickup still pending`, with 297 recipients, 7 purchasers, and a conversion rate of 2.36%. Another strong example was campaign `32` with topic `order ready for pickup`, which had 356 purchasers and a small but visible social support share of 1.12%.

This suggests that transactional campaigns were more effective than bulk campaigns in this dataset. It also suggests that social links may help in some cases, but the effect is not large in most of the top campaigns.

Table 1: Q1 execution summary

Database	Time (sec)	Rows returned
PostgreSQL	18.24	20
MongoDB	237.08	20

4.1.2 Q2: Personalized product recommendation

Q2 builds product recommendations for the home page. The idea is simple: take the top 100 active users, look at the products their friends interacted with, give a higher weight to purchases and cart actions, remove products that the target user already touched, and keep the top 5 products for each target user.

Both PostgreSQL and MongoDB returned 140 recommendation rows, which means the logic was consistent between the two systems. One clear example is user `550133755`, for whom product `1005124` had the highest score of 59 based on 37 friend interactions. Many top recommendations were smartphones and electronics products, which shows that these categories were very active in the dataset.

The result is useful for the business because it gives a practical way to show social-based recommendations on the home page.

Table 2: Q2 execution summary

Database	Time (sec)	Rows returned
PostgreSQL	4.07	140
MongoDB	3.41	140

4.1.3 Q3: Full-text search on top product categories

Q3 takes the top recommended products from Q2, extracts common keywords from their `category_code`, and then searches products using these keywords. In MongoDB, the keywords used were `appliances`, `electronics`, and `smartphone`. MongoDB returned 60 rows for this task and did it very fast.

In PostgreSQL, the stored analysis result for Q3 returned 0 rows. So, for this dataset and implementation, the MongoDB text search gave useful output, while the PostgreSQL version did not return matching products in the saved run. This is an important difference between the two systems in this project.

Table 3: Q3 execution summary

Database	Time (sec)	Rows returned
PostgreSQL	3.75	0
MongoDB	0.50	60

4.2 Benchmarking

The benchmark data was saved in:

- `benchmark_results/postgres_results.csv`,
- `benchmark_results/mongo_results.csv`.

According to the assignment, each query should be run 5 times. This was completed for PostgreSQL. For MongoDB, I ran each query only one time because the first query was very slow on my laptop. This limitation is important, so the MongoDB benchmark numbers should be read with care.

4.2.1 Raw benchmark results

Table 4: Raw benchmark times for PostgreSQL

Query	Run 1	Run 2	Run 3	Run 4	Run 5
Q1	36686.850	47662.177	23926.330	16823.853	15065.040
Q2	5485.536	3507.523	3526.006	3806.173	3713.116
Q3	4206.594	4181.127	4478.622	4828.611	4522.320

Table 5: Raw benchmark times for MongoDB

Query	Run 1	Note
Q1	227880.041	only one run
Q2	3833.373	only one run
Q3	290.049	only one run

4.2.2 Average and standard deviation

The MongoDB standard deviation is zero only because there was one run, not because the performance is perfectly stable.

Table 6: Benchmark summary statistics

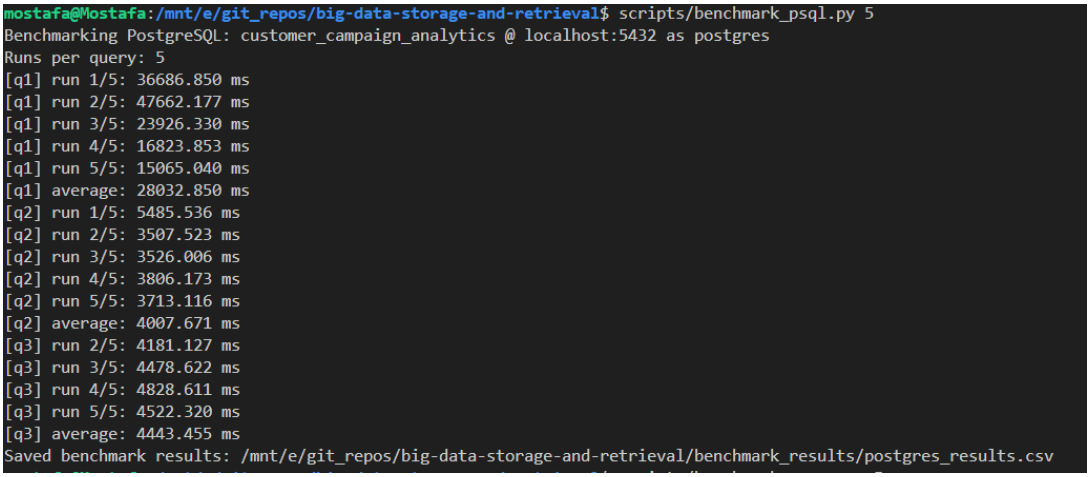
Database	Query	Runs	Average (ms)	Std. dev. (ms)
PostgreSQL	Q1	5	28032.850	13882.286
PostgreSQL	Q2	5	4007.671	835.703
PostgreSQL	Q3	5	4443.455	264.915
MongoDB	Q1	1	227880.041	0.000
MongoDB	Q2	1	3833.373	0.000
MongoDB	Q3	1	290.049	0.000

Average execution times (ms)
PostgreSQL: Q1 = 28032.850, Q2 = 4007.671, Q3 = 4443.455
MongoDB: Q1 = 227880.041, Q2 = 3833.373, Q3 = 290.049

Figure 4: Text summary of average query execution times

4.2.3 Execution time chart

4.2.4 Benchmark screenshots



```
mostafa@Mostafa:/mnt/e/git_repos/big-data-storage-and-retrieval$ scripts/benchmark_psql.py 5
Benchmarking PostgreSQL: customer_campaign_analytics @ localhost:5432 as postgres
Runs per query: 5
[q1] run 1/5: 36686.850 ms
[q1] run 2/5: 47662.177 ms
[q1] run 3/5: 23926.330 ms
[q1] run 4/5: 16823.853 ms
[q1] run 5/5: 15065.040 ms
[q1] average: 28032.850 ms
[q2] run 1/5: 5485.536 ms
[q2] run 2/5: 3507.523 ms
[q2] run 3/5: 3526.006 ms
[q2] run 4/5: 3806.173 ms
[q2] run 5/5: 3713.116 ms
[q2] average: 4007.671 ms
[q3] run 2/5: 4181.127 ms
[q3] run 3/5: 4478.622 ms
[q3] run 4/5: 4828.611 ms
[q3] run 5/5: 4522.320 ms
[q3] average: 4443.455 ms
Saved benchmark results: /mnt/e/git_repos/big-data-storage-and-retrieval/benchmark_results/postgres_results.csv
```

Figure 5: PostgreSQL benchmark screenshot

5 Discussion

The results show clear strengths and weaknesses for the two tested systems.

PostgreSQL was strong for structured relational analysis. Its schema was normalized, clear, and stable. Q1 and Q2 worked correctly, and the benchmark results were based on five runs, which gives a more reliable view of its performance. The main weakness in this work was Q3, where the saved run returned no rows.

MongoDB was flexible and performed very well for the text search task. Q3 was especially strong in MongoDB, both in result quality and speed. Q2 was also slightly faster than PostgreSQL. However, MongoDB had a major weakness in Q1. That query needed a large amount of processing over messages and friendship links, and on the test laptop it was much slower than PostgreSQL.

The campaign analysis also gives a business insight. Transactional email campaigns had the best conversion rates in the top results. Bulk campaigns appeared in the ranking, but they were

```
mostafa@Mostafa:/mnt/e/git_repos/big-data-storage-and-retrieval$ scripts/benchmark_mongo.py
Benchmarking MongoDB: customer_campaign_analytics @ mongodb://localhost:27017
Runs per query: 1
[23:29:22] q1: ensuring helper indexes.
[23:29:22] q1: building campaign-user purchase table.
[23:33:23] q1: campaign-user table ready (2298764 rows).
[23:33:23] q1: loading friendship graph into memory.
[23:33:36] q1: friendship graph loaded (656455 users with at least one friend).
[23:33:36] q1: computing social purchaser counts.
[q1] run 1/1: 227880.041 ms
[q1] average: 227880.041 ms
[23:33:37] q2: selecting top 100 active users from events.
[23:33:38] q2: found 100 target users, running recommendation aggregation.
[23:33:40] q2: aggregation finished.
[q2] run 1/1: 3833.373 ms
[q2] average: 3833.373 ms
[q3] Keywords from latest q2 result: ['electronics', 'smartphone', 'appliances']
[23:33:40] q3: ensuring text index on products (category_code, brand).
[23:33:40] q3: searching products for keyword 'electronics'.
[23:33:41] q3: searching products for keyword 'smartphone'.
[23:33:41] q3: searching products for keyword 'appliances'.
[q3] run 1/1: 290.049 ms
[q3] average: 290.049 ms
Saved benchmark results: /mnt/e/git_repos/big-data-storage-and-retrieval/benchmark_results/mongo_results.csv
```

Figure 6: MongoDB benchmark screenshot

not the strongest ones. The social support signal was present, but usually small, so it should be treated as an extra feature and not the only rule for future targeting.

Neo4j is still worth mentioning. In theory, it is a natural choice for friendship-based recommendation because graph traversal fits that problem well. Still, the technical issues I faced meant I could not finish the Neo4j analysis and benchmark stages. Because of that, I cannot make a fair performance comparison for Neo4j in this report.

6 Conclusion

In this assignment, three data models were built for the same e-commerce dataset. PostgreSQL and MongoDB were fully used for the analysis tasks, while Neo4j was modeled but not fully tested because of technical issues. PostgreSQL gave better stability and much better Q1 performance. MongoDB gave the best Q3 result and the fastest text search. Based on the completed experiments, PostgreSQL looks like the safer general choice for this workload, while MongoDB is a strong option when flexible document storage and text search are important.